# AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 11: Stochastic Gradient Descent

Adam Prugel-Bennett

November 2, 2026

## 1 Keywords

- SGD, momentum, Nesterov, RMSProp, Adam, AdamW, learning-rate schedules, loss landscapes

## 2 Main Points

### 2.1 Optimisation at Scale

- Deep networks have millions to billions of parameters. A Hessian with $10^9$ parameters would have $10^{18}$ entries, so Newton and quasi-Newton methods are out; we use gradients only
- **Mini-batches.** Rather than the gradient of the loss over the whole dataset $\mathcal{D}$ we use a random mini-batch $\mathcal{B} \subset \mathcal{D}$, $|\mathcal{B}| \ll |\mathcal{D}|$:

$$\boldsymbol{\nabla} L_{\mathcal{B}}(\boldsymbol{w}) = \frac{1}{|\mathcal{B}|} \sum_{(\boldsymbol{x},y)\in\mathcal{B}} \boldsymbol{\nabla} L(\boldsymbol{x}, y|\boldsymbol{w})$$

  Averaging (rather than summing) makes this an unbiased estimate of the full gradient $\boldsymbol{\nabla} L_{\mathcal{D}}$. Following it is *stochastic gradient descent* (SGD)

  - Each step is much cheaper, so we can take many small steps, and over many steps the random errors average out
- **Automatic differentiation** computes gradients of almost any program, which is what made very large models practical
- **Why the slower asymptotic convergence doesn't matter much**
  - We spend most of the time far from a quadratic minimum, where the asymptotic rates do not apply
  - With ReLUs the loss is not twice differentiable, so the Taylor argument behind Newton's method breaks down
  - We really want low generalisation error; the training loss is only a surrogate, so its exact minimum is not that important

### 2.2 One Step Size, Many Curvatures

- From the last lecture: along the $i$-th eigenvector of the Hessian a gradient step multiplies the error by $1 - r\,\lambda_i$, so we need $r < 2/\lambda_{\max}$, and the flattest directions then shrink by at best $1 - 2/\kappa$ per step, with $\kappa$ the condition number
- Too large an $r$ diverges, quickly giving NaNs. Everything in this lecture is a way round this conflict

## 2.3 Momentum

- In a narrow valley gradient descent steps consistently along the flat direction but bounces across the steep one. Momentum remembers earlier steps

$$\boldsymbol{v}^{(t+1)} = \gamma\, \boldsymbol{v}^{(t)} - r\, \boldsymbol{\nabla} L_{\mathcal{B}}(\boldsymbol{w}^{(t)})$$
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \boldsymbol{v}^{(t+1)}$$

  with $0 < \gamma < 1$ (typically $0.9$): how much velocity is kept, so $1 - \gamma$ acts as friction

- Where the gradient keeps the same sign the steps add up, to an effective step $r/(1-\gamma)$ (ten times larger for $\gamma = 0.9$, Exercise 3). Where it alternates they cancel

- **Nesterov momentum** takes the gradient where momentum is about to carry us,

$$\boldsymbol{v}^{(t+1)} = \gamma\, \boldsymbol{v}^{(t)} - r\, \boldsymbol{\nabla} L\big(\boldsymbol{w}^{(t)} + \gamma\, \boldsymbol{v}^{(t)}\big)$$

  - With well-tuned momentum (heavy-ball or Nesterov) the error factor per step on a quadratic improves from about $1 - 2/\kappa$ to about $1 - 2/\sqrt{\kappa}$: the number of steps scales as $\sqrt{\kappa}$ rather than $\kappa$. For $\kappa = 10^4$ that is roughly 50 steps per factor $e$ instead of $5000$
  - Convergence is still *linear*: momentum fixes the condition number, not the order of convergence. Super-linear convergence needs curvature information (BFGS, last lecture)
  - On general convex problems Nesterov's method reaches error $O(1/t^2)$ after $t$ steps, against $O(1/t)$ for gradient descent; no method that only uses gradients can do better. The optional module COMP6260, Optimisation for Machine Learning, covers this properly

## 2.4 Adaptive Methods

- Different parameters need different step sizes. Adaptive methods rescale the step for each parameter separately, which roughly makes the valleys more spherical

- **RMSProp** keeps a running average of the squared gradient for each parameter and divides by its square root

$$S_i^{(t+1)} = (1-\gamma)\, S_i^{(t)} + \gamma \left(\frac{\partial L_{\mathcal{B}}}{\partial w_i}\right)^2$$
$$w_i^{(t+1)} = w_i^{(t)} - \frac{r}{\sqrt{S_i^{(t+1)}} + \epsilon} \frac{\partial L_{\mathcal{B}}}{\partial w_i}$$

  Parameters with large gradients take small steps and those with small gradients large ones; each moves about $r$ per step, whatever the scale of the loss

- **Adam** (adaptive moment estimation) is RMSProp plus momentum. It keeps running averages of the gradient and of its square,

$$M_i^{(t+1)} = (1-\beta)\, M_i^{(t)} + \beta\, \frac{\partial L_{\mathcal{B}}}{\partial w_i}, \qquad S_i^{(t+1)} = (1-\gamma)\, S_i^{(t)} + \gamma \left(\frac{\partial L_{\mathcal{B}}}{\partial w_i}\right)^2$$

  - Both start at zero, so early on they are biased towards zero. After $t+1$ updates a constant input $x$ gives only $(1-(1-\beta)^{t+1})\, x$ (Exercise 1), so Adam corrects them:

$$\hat{M}_i^{(t+1)} = \frac{M_i^{(t+1)}}{1-(1-\beta)^{t+1}} \qquad\qquad \hat{S}_i^{(t+1)} = \frac{S_i^{(t+1)}}{1-(1-\gamma)^{t+1}}$$

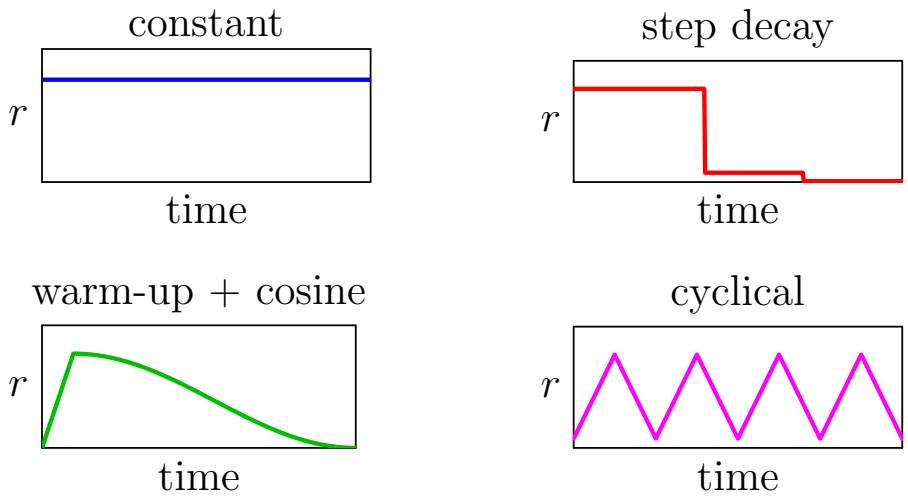
- The update is
$$w_i^{(t+1)} = w_i^{(t)} - \frac{r}{\sqrt{\hat{S}_i^{(t+1)}} + \epsilon} \hat{M}_i^{(t+1)}$$
- In this notation the usual defaults are $\beta = 0.1$ and $\gamma = 0.001$ (the literature writes $\beta_1 = 1 - \beta = 0.9$ and $\beta_2 = 1 - \gamma = 0.999$)
- Adam and its variants are very robust: a network that will not train with SGD often trains with Adam

- **AdamW.** Weight decay pulls the weights towards zero. Adding $\frac{\eta}{2} \|\boldsymbol{w}\|^2$ to the loss adds $\eta\, w_i$ to each gradient, which Adam then divides by $\sqrt{\hat{S}_i}$, so weights with large gradients are hardly decayed at all. AdamW applies the decay to the weights directly, $w_i \leftarrow w_i - r\,\eta\, w_i$, outside the adaptive rescaling (Exercise 4). It is the default for training transformers

- **Covariance.** Standard vector operations (addition, scalar and matrix multiplication) give results that do not depend on the coordinate system: rotate the coordinates and you get the rotated answer. Element-wise operations do not have this property
  - Adaptive methods rescale each coordinate separately, so they are not invariant to rotations
  - If two weights interact, the curvature can be high along $\boldsymbol{e}_i + \boldsymbol{e}_j$ and low along $\boldsymbol{e}_i - \boldsymbol{e}_j$. Rescaling $w_i$ and $w_j$ separately cannot equalise this

## 2.5 Learning-Rate Schedules

- In practice the learning rate is changed during training




  - Large $r$ early: the noisy steps cross small valleys and find the big ones
  - Small $r$ late: settle into a valley. Dividing $r$ by ten when progress stalls often gives a sudden improvement (step decay)
  - Cosine decay lowers $r$ smoothly; cyclical schedules repeatedly raise and lower it
  - A short *warm-up* at the start stops the first steps diverging before Adam's running averages have settled; common when training transformers

## 2.6 Loss Landscapes

- We minimise over an enormous space. Unless the loss is convex there will be many local minima, and for non-convex losses no efficient algorithm is guaranteed to find the global minimum. In such large spaces we may never reach even a local minimum

- Mini-batch noise means the weights keep moving; a smaller step averages over more batches and follows the true gradient more closely
- **Symmetries**
  - Permuting the neurons of a layer (or the feature maps of a CNN), together with their weights, gives the same network: $n!$ equivalent solutions for $n$ hidden units
  - For odd activations such as $\tanh$, flipping the sign of a unit's input and output weights also changes nothing
  - There are continuous symmetries too: in a linear network $\mathbf{W}_2\mathbf{W}_1 = (\mathbf{W}_2\mathbf{U})(\mathbf{U}^{-1}\mathbf{W}_1)$ for any invertible $\mathbf{U}$
  - So there are huge sets of weights with the same loss. Between two equivalent minima there must be a saddle point
  - Skip connections (ResNets, transformers) break the permutation symmetry, which seems to make optimisation easier

# 3 Exercises

1. **Removing the start-up bias.** Consider a running average

$$a^{(t+1)} = (1-\gamma)\, a^{(t)} + \gamma\, x^{(t)}$$

   with constant input $x^{(t)} = x$ and $a^{(0)} = 0$

   (a) Write $a^{(t)}$ as a sum

   (b) Using the sum of a geometric series,

$$\sum_{i=0}^{t-1} z^i = 1 + z + \cdots + z^{t-1} = \frac{1-z^t}{1-z}$$

   write $a^{(t)}$ in closed form

   (c) Find the correction that makes the corrected average equal $x$ for all $t$

2. **Gradient descent in a quadratic minimum.** Minimise

$$f(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T}\mathbf{Q}(\boldsymbol{x} - \boldsymbol{x}^*)$$

   using $\boldsymbol{x}(t+1) = \boldsymbol{x}(t) - r\,\boldsymbol{\nabla} f(\boldsymbol{x})$

   (a) Write down the update equation

   (b) Subtract $\boldsymbol{x}^*$ from both sides and write down an update equation for $\boldsymbol{x}(t) - \boldsymbol{x}^*$

   (c) Write a closed form solution for $\boldsymbol{x}(t) - \boldsymbol{x}^*$

   (d) Using the eigen-decomposition $\mathbf{Q} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$ rewrite the closed form solution

   (e) Defining $\boldsymbol{u}(t) = \mathbf{V}^\mathsf{T}(\boldsymbol{x}(t) - \boldsymbol{x}^*)$ write the update equation for the components $u_i(t)$ and explain what happens when $\lambda_i > 2/r$

3. **Momentum adds up.** Suppose the gradient is the same, $\boldsymbol{g}$, at every step. Starting from $\boldsymbol{v}^{(0)} = \mathbf{0}$, find $\boldsymbol{v}^{(t)}$ and its limit as $t \to \infty$

4. **Why AdamW?** With the loss $L(\boldsymbol{w}) + \frac{\eta}{2}\|\boldsymbol{w}\|^2$

   (a) Show that plain gradient descent on this loss is the same as gradient descent on $L$ followed by $w_i \leftarrow w_i - r\,\eta\, w_i$

   (b) Explain why this is no longer true for Adam

# 4 Experiments

## 4.1 Gradient descent, momentum and Nesterov

- On a quadratic $f(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^{\mathsf{T}}\mathbf{H}\boldsymbol{w}$ with a random positive definite Hessian, compare the methods with the same step size

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(5, 4))
H = X.T @ X                          # Hessian of f(w) = w^T H w / 2
lam = np.linalg.eigvalsh(H)
print("eigenvalues:", np.round(lam, 2), " kappa =", round(lam[-1] / lam[0]))

f = lambda w: w @ H @ w / 2
grad = lambda w: H @ w
w0 = rng.normal(size=4)
r = 1 / lam[-1]                      # safely below 2 / lambda_max

def run(gamma, nesterov=False, steps=200):
    w, v = w0.copy(), np.zeros(4)
    for t in range(steps):
        g = grad(w + gamma * v) if nesterov else grad(w)
        v = gamma * v - r * g
        w = w + v
    return f(w)

print("gradient descent:", run(0.0))
print("momentum       :", run(0.9))
print("Nesterov       :", run(0.9, nesterov=True))
print("r = 2.1/lambda_max diverges:", end=" ")
r = 2.1 / lam[-1]; print(run(0.0))
```

- Here $\kappa \approx 39$. After 200 steps gradient descent reaches $f \approx 5 \times 10^{-6}$, momentum $10^{-10}$ and Nesterov $10^{-14}$; just over the stability limit gradient descent blows up. Try other values of `gamma` and `r`

# 5 Solutions

1. Removing the start-up bias

   (a) Writing $a^{(t)}$ as a sum

   $$a^{(1)} = (1-\gamma)\,a^{(0)} + \gamma\,x = \gamma\,x$$
   $$a^{(2)} = (1-\gamma)\,a^{(1)} + \gamma\,x = (1-\gamma)\,\gamma\,x + \gamma\,x$$
   $$a^{(t)} = \gamma\,x \sum_{i=0}^{t-1} (1-\gamma)^i$$

   (b) To prove the identity, multiply the series by $1-z$: $(1-z)(1+z+\cdots+z^{t-1}) = 1-z^t$. Then

   $$a^{(t)} = \gamma\,x\,\frac{1-(1-\gamma)^t}{1-(1-\gamma)} = x\left(1-(1-\gamma)^t\right)$$

   which only approaches $x$ as $t \to \infty$

(c) Divide by $1-(1-\gamma)^t$:

$$\bar{a}^{(t)} = \frac{a^{(t)}}{1-(1-\gamma)^t} = x$$

Adam's $\hat{M}^{(t+1)}$ and $\hat{S}^{(t+1)}$ are this correction after $t+1$ updates

2. Gradient descent in a quadratic minimum

   (a) $\boldsymbol{x}(t+1) = \boldsymbol{x}(t) - r\,\mathbf{Q}(\boldsymbol{x}(t) - \boldsymbol{x}^*)$

   (b) $\boldsymbol{x}(t+1) - \boldsymbol{x}^* = (\mathbf{I} - r\,\mathbf{Q})(\boldsymbol{x}(t) - \boldsymbol{x}^*)$

   (c) $\boldsymbol{x}(t) - \boldsymbol{x}^* = (\mathbf{I} - r\,\mathbf{Q})^t(\boldsymbol{x}(0) - \boldsymbol{x}^*)$

   (d) $\boldsymbol{x}(t) - \boldsymbol{x}^* = \mathbf{V}(\mathbf{I} - r\,\mathbf{\Lambda})^t\mathbf{V}^\mathsf{T}(\boldsymbol{x}(0) - \boldsymbol{x}^*)$, because $\mathbf{I} - r\,\mathbf{Q} = \mathbf{V}(\mathbf{I} - r\,\mathbf{\Lambda})\mathbf{V}^\mathsf{T}$ and in its $t$-th power every inner $\mathbf{V}^\mathsf{T}\mathbf{V}$ is $\mathbf{I}$

   (e) $u_i(t) = (1 - r\,\lambda_i)^t\,u_i(0)$. If $\lambda_i > 2/r$ then $|1 - r\lambda_i| > 1$ and $u_i(t)$ grows exponentially: any component along $\boldsymbol{v}_i$ diverges if the curvature in that direction exceeds $2/r$

3. $\boldsymbol{v}^{(1)} = -r\boldsymbol{g}$, $\boldsymbol{v}^{(2)} = -r\boldsymbol{g}(1+\gamma)$, and in general $\boldsymbol{v}^{(t)} = -r\,\boldsymbol{g}\,(1-\gamma^t)/(1-\gamma)$, which tends to $-r\,\boldsymbol{g}/(1-\gamma)$: an effective step size $r/(1-\gamma)$

4. Why AdamW

   (a) The gradient of the penalty is $\eta\,\boldsymbol{w}$, so the step is $\boldsymbol{w} - r\,\boldsymbol{\nabla}L - r\,\eta\,\boldsymbol{w}$: a gradient step on $L$ plus the decay $w_i \leftarrow w_i - r\,\eta\,w_i$

   (b) Adam divides each component of the gradient, including $\eta\,w_i$, by $\sqrt{\hat{S}_i}$ (and averages it into the momentum). Weights with large gradients get large $\hat{S}_i$ and so very little decay. AdamW decays every weight by the same fraction $r\,\eta$